



Universidad de Granada

DOBLE GRADO EN INGENIERÍA INFORMÁTICA Y
MATEMÁTICAS

METAHEURÍSTICAS

Práctica 1:

*Técnicas de Búsqueda de Poblaciones para el Problema
de Asignación Con Restricciones (PAR)*

Autor: Jesús Muñoz Velasco

DNI: 76668409W

E-mail: jesusmuve18@correo.ugr.es

Horario prácticas: Miércoles 15:30 - 17:30 (Grupo A3)

Curso 2025-2026

Índice

1. Descripción del problema	2
2. Aplicación de los algoritmos empleados	3
2.1. Esquema de representación de soluciones	3
2.2. Esquema de representación de restricciones	3
2.2.1. Forma Matricial	3
2.2.2. Forma Vectorial	3
2.3. Criterio de validación de Soluciones	4
2.4. Reparación de soluciones no válidas	4
2.5. Evaluación de una solución (función objetivo)	4
2.5.1. Cálculo de centroides de una solución	4
2.5.2. Cálculo la desviación de una solución	5
2.5.3. Cálculo la infactibilidad de una solución	5
2.5.4. Evaluación de una solución	5
3. Estructura del método de búsqueda	6
3.1. Algoritmo aleatorio	6
3.2. Algoritmo Greedy	6
3.2.1. Función Heurística (Incremento del coste)	6
3.2.2. Estructura del código Greedy	6
3.3. Algoritmo de Búsqueda Local	7
3.3.1. Generación del vecindario	7
3.3.2. Factorización de la solución	8
3.3.3. Estructura del código de Búsqueda Local	9
4. Estructura general del código	10
5. Experimentos y análisis de resultados	11
5.1. Descripción de casos del problema	11
5.2. Resultados obtenidos	11
5.2.1. Tablas globales por problema	12
5.2.2. Boxplots globales por problema	13
5.3. Análisis de resultados	14
5.3.1. Calidad de las soluciones	14
5.3.2. Satisfacción de restricciones	15
5.3.3. Eficiencia computacional	15
5.3.4. Impacto de la escalabilidad y las restricciones	15

1. Descripción del problema

El Problema de Agrupamiento con Restricciones (PAR), también conocido como *Constrained Clustering*, consiste en la partición de un conjunto de n datos o instancias en k grupos (clústeres) disjuntos, basándose en la similitud de sus características y cumpliendo en la mayor medida posible un conjunto de restricciones impuestas sobre los pares de instancias.

Formalmente, sea $X = \{x_1, x_2, \dots, x_n\}$ un conjunto de n elementos en un espacio de dimensión d . El objetivo es encontrar una partición $C = \{C_1, C_2, \dots, C_k\}$ tal que se minimice la dispersión intra-clúster (medida como la distancia euclídea de cada elemento al centroide de su clúster), sujeta a un conjunto de restricciones a nivel de instancia (de tipo Must-Link y Cannot-Link).

Hay 2 tipos de restricciones posibles sobre 2 instancias $x_i, x_j \in X$:

-) *Must-Link* (ML). Los elementos x_i y x_j deben pertenecer al mismo clúster.
-) *Cannot-Link* (CL). Los elementos x_i y x_j no pueden pertenecer al mismo clúster.

Dado que encontrar una solución factible que satisfaga todas las restricciones es un problema NP-completo en la mayoría de configuraciones prácticas, relajaremos el incumplimiento de estas condiciones, donde las violaciones se penalizan en la función objetivo. La función objetivo a minimizar se define como

$$fitness(solucion) = desviacion(solucion) + \lambda \cdot infeasibilidad(solucion)$$

donde establecemos $\lambda = \frac{max_{dist}(X)}{restricciones(R)}$ para asegurar que la infactibilidad y la desviación tienen la suficiente relevancia. Además, la desviación de una solución se define como

$$desviacion(C_s) = \frac{1}{k} \sum_{C_i \in C_s} \frac{1}{|C_i|} \sum_{\vec{x}_j \in C_i} \left\| \vec{x}_j - \frac{1}{|C_i|} \sum_{x_j \in C_i} \vec{X}_j \right\|$$

y la infactibilidad como el número de condiciones incumplidas de una solución.

2. Aplicación de los algoritmos empleados

Para la resolución del PAR mediante los algoritmos Aleatorio, Greedy y Búsqueda Local, es necesario establecer un marco de trabajo común que defina cómo se representan, evalúan y manipulan las soluciones.

2.1. Esquema de representación de soluciones

Una solución S se representa como un vector de tamaño n (número de instancias), donde el índice i representa la instancia x_i , y el valor almacenado en $S[i]$ indica el índice del clúster al que ha sido asignada.

$$S = [c_0, c_1, \dots, c_{n-1}] \quad \text{donde} \quad c_i \in \{0, 1, \dots, k-1\}$$

Esta representación garantiza que toda instancia pertenezca a un clúster y solo uno.

2.2. Esquema de representación de restricciones

Para representar el conjunto de restricciones R se pueden usar dos estrategias principalmente, según convenga para mayor eficiencia en las evaluaciones de una solución:

2.2.1. Forma Matricial

El conjunto R se representa como una matriz de dimensión $n \times n$ donde cada elemento R_{ij} representará la restricción existente entre la instancia x_i y la x_j . Podrá tomar los siguientes valores:

-) $R_{ij} = 1$: Indica una restricción *Must-Link*.
-) $R_{ij} = -1$: Indica una restricción *Cannot-Link*.
-) $R_{ij} = 0$: Indica que no existe ninguna restricción entre x_i y x_j .

Por la naturaleza de esta matriz será cuadrada, simétrica y los elementos de la diagonal principal no podrán valer -1 . Es muy útil para saber si 2 elementos tienen alguna restricción entre ellos.

2.2.2. Forma Vectorial

En este caso se ha creado un nuevo tipo de dato llamado “Restricción” y tendrá la siguiente estructura:

```
1 struct Restriccion:
2     i, j          // Índices de los puntos  $x_i$  y  $x_j$  que participan en la restricción
3     tipo          // 1 para Must-Link, -1 para Cannot-Link
```

Código Fuente 1: Estructura de una restricción vectorial

De esta forma, el conjunto de restricciones R será un vector de “Restricciones” según acabamos de describir. Aunque esta estructura sea peor para saber si 2 elementos tienen alguna restricción, es mucho mejor para recorrer las restricciones una por una para cálculos como la infactibilidad.

2.3. Criterio de validación de Soluciones

Dado que el problema establece que hay un número k de clústeres para la solución se considerará que una solución S es válida si y solo si aparecen todos los clústeres en la solución, es decir, si se verifica que

$$c_i \in S \quad \forall c_i \in \{0, 1, \dots, k-1\}$$

2.4. Reparación de soluciones no válidas

El proceso de reparación de una solución S con clústeres vacíos consistirá en seleccionar un elemento $c_0 \in S$ que aparezca al menos 2 veces en el vector (para no dejar dicho clúster vacío) y sustituir su valor por el índice del clúster vacío. Se repite este proceso hasta que no haya clústeres vacíos. Seguiría el siguiente esquema:

```

1  Función RepararSolucion(S):
2      // Inicializamos el vector n_puntos con el número de puntos de cada clúster
   // en la solución S
3      n_puntos = nPuntosClusteres(S)
4
5      Para cada clúster c desde 0 hasta k-1:
6          Si n_puntos[c] == 0:
7              arreglado = Falso
8              instancia = 1
9
10             // Buscar una instancia de un clúster con más de 1 elemento
11             Mientras arreglado == Falso y instancia ≤ n:
12                 cluster_actual = S[instancia]
13
14                 Si n_puntos[cluster_actual] > 1:
15                     // Reasignar al clúster vacío
16                     S[instancia] = c_vacio
17                     n_puntos[cluster_actual] = n_puntos[cluster_actual] - 1
18                     n_puntos[c_vacio] = n_puntos[c_vacio] + 1
19                     arreglado = Verdadero
20
21                 instancia++
22
23     return S

```

Código Fuente 2: Pseudocódigo de la reparación de una solución

2.5. Evaluación de una solución (función objetivo)

La evaluación de una solución requiere calcular los centroides de los clústeres a partir de las asignaciones actuales en S , calcular la desviación media de cada clúster y contabilizar las restricciones violadas consultando la matriz R . Esto se hará de la siguiente forma:

2.5.1. Cálculo de centroides de una solución

```

1  Funcion obtenerCentroides(S):
2      n_puntos = nPuntosClusteres(S)
3
4      // Recorro todos los vectores sumándolos a su cluster
5      Para cada instancia i de X:
6          cluster = S[i]
7          Para cada dimension d de la instancia i:
8              centroides[cluster].coordenada[d] += X[i].coordenada[d]
9

```

```

10
11 // Divido entre el número de cada cluster para calcular la media
12 Para cada cluster c desde 0 hasta k-1
13     Para cada dimension d del centroide centroides[c]:
14         centroides[c].coordenada[d] /= n_puntos[c]
15
16 return centroides

```

Código Fuente 3: Pseudocódigo del cálculo de los centroides de una solución

2.5.2. Cálculo la desviación de una solución

```

1 Función Desviacion(S):
2     centroides = obtenerCentroides(S)
3     n_puntos = nPuntosClusteres(S)
4
5     // Sumamos las distancias
6     Para cada instancia i de X:
7         cluster = S[i]
8         distanciaic[cluster] += distancia(X[i], centroides[cluster])
9
10    desviacion = 0
11
12    // Sumamos la media de las distancias
13    Para cada clúster c desde 0 hasta k-1
14        Si n_puntos[c] > 0:
15            desviacion += (distanciaic[c]/n)
16
17    return (desviacion/k)

```

Código Fuente 4: Pseudocódigo del cálculo de la desviación de una solución

2.5.3. Cálculo la infactibilidad de una solución

```

1 Función Infactibilidad(S):
2     inf = 0
3
4     Para cada restricción res en el conjunto de restricciones R
5         juntos = (S[res.i] == S[res.j])
6
7         Si res es de tipo ML y !juntos:
8             inf++ // Error: Deberían estar juntos (Must-Link)
9         Sino si res es de tipo CL y juntos:
10            inf++ // Error: No deberían estar juntos (Cannot-Link)
11
12    return inf

```

Código Fuente 5: Pseudocódigo del cálculo de la infactibilidad de una solución

2.5.4. Evaluación de una solución

Finalmente, usando las funciones previamente descritas, y previamente fijado el valor de lambda, la función objetivo se calculará de la siguiente forma

```

1 Evaluacion(S) {
2     C = Desviacion(S)
3     inf = Infactibilidad(S)
4
5     return (C + lambda * inf)
6 }

```

Código Fuente 6: Pseudocódigo del cálculo de la función objetivo

3. Estructura del método de búsqueda

A continuación se describen más a fondo cada algoritmo utilizado junto con una descripción de las funciones propias que estos usan.

3.1. Algoritmo aleatorio

El algoritmo aleatorio genera múltiples soluciones independientes, evalúa cada una usando la función objetivo descrita previamente, y devuelve la mejor encontrada tras un número determinado de iteraciones. El código sigue la siguiente estructura:

```
1  optimizarAleatorio(maxevals):
2      mejor_fitness = -1
3      mejor_solucion = null
4
5      Para cada iteracion i desde 0 hasta maxevals:
6          S = generarSolucionAleatoria()
7          fitness = Evaluacion(S)
8
9          Si fitness < mejor_fitness o mejor_fitness < 0:
10             mejor_solucion = S
11             mejor_fitness = fitness
12
13     return mejor_solucion
```

Código Fuente 7: Estructura del algoritmo aleatorio

3.2. Algoritmo Greedy

El algoritmo Greedy construye la solución paso a paso. Comienza con una solución generada aleatoriamente. Comienza generando una solución aleatoria que sea válida (si no lo es se arregla como se ha descrito en la sección anterior). Posteriormente se realiza un proceso iterativo en el que se prueba a modificar la solución anterior probando en cada punto del vector solución qué ocurriría al asignarlo a cada clúster. De entre estas modificaciones virtuales se elegirá aquella que minimice el número de restricciones incumplidas y en caso de empate se elegirá la que minimice la desviación general. Este proceso se realizará hasta que no se encuentre una configuración mejor modificando un único elemento.

3.2.1. Función Heurística (Incremento del coste)

La heurística en este caso valora el coste de añadir la instancia x_i al clúster c . Se calcula la distancia al centroide actual del clúster y las penalizaciones por restricciones generadas con las instancias ya asignadas (función evaluación anterior).

3.2.2. Estructura del código Greedy

Con la heurística ya mencionada, el algoritmo seguirá la siguiente estructura

```
1  optimizarGreedy(maxevals):
2
3      // Generamos aleatoriamente centroides en el dominio del problema
4      centroides = generarCentroidesAleatorios()
5
6      // Generamos una solución aleatoria
7      S = generarSolucionAleatoria()
```

```

8
9     Si S no es válida:
10         S = RepararSolucion(S)
11
12     // Generamos y barajamos los índices
13     RSI = generarIndices(0,n-1)
14     barajar(RSI)
15
16     Repetir:
17         modificado = false
18
19         Para cada índice i de RSI:
20             cluster_actual = S[i]
21             mejor_cluster = currentCluster
22             mejor_inf = infinito
23             mejor_d = infinito
24
25             // Evaluar todos los posibles clústeres para el punto  $x_i$ 
26             Para cada índice j desde 0 hasta k-1:
27                 inf = Evaluacion(S)
28                 d = distancia(X[i], centroides[j])
29
30                 Si inf < mejor_inf:
31                     mejor_inf = inf
32                     mejor_d = d
33                     mejor_cluster = j
34                 Sino si inf == mejor_inf y d < mejor_d:
35                     mejor_d = d
36                     mejor_cluster = j
37
38             // Si hay un clúster mejor para este punto
39             Si mejor_cluster != cluster_actual:
40                 S[i] = mejor_cluster
41                 modificado = true
42
43     Mientras modificado==true
44
45     return S

```

Código Fuente 8: Estructura del algoritmo Greedy

3.3. Algoritmo de Búsqueda Local

El algoritmo de Búsqueda Local parte de una solución inicial (generada aleatoriamente) y explora su vecindario aplicando pequeñas modificaciones. Si un vecino mejora la función objetivo, reemplaza a la solución actual. El proceso se detiene cuando ningún vecino mejora la solución (óptimo local) o se alcanza el límite de evaluaciones. Para ello se sirve de las siguientes funciones:

3.3.1. Generación del vecindario

Dada una solución S , se genera su vecindario, entendiendo este como los cambios posibles de clúster, que serán factibles siempre y cuando cumplan con no dejar un clúster vacío. Se hará de la siguiente forma:

```

1     generarVecindario(S):
2         Para cada i desde 0 hasta n-1:
3             cluster_actual = S[i]
4
5             // Sólo guardamos los factibles
6             Si nPuntosCluster(cluster_actual) > 1:
7                 Para cada índice j desde 0 hasta k-1:
8                     Si j!=cluster_actual:
9                         vecindario.añadir({i, j})

```

```

10
11     return vecindario

```

Código Fuente 9: Pseudocódigo de la generación del vecindario

3.3.2. Factorización de la solución

Dado que evaluar la función objetivo completa tiene un coste computacional alto se implementa una factorización de la solución donde se tienen precalculados ciertos valores. Al mover x_i del c_{origen} al $c_{destino}$ tan solo se producen cambios en estos 2 clústeres por lo que solo habrá que comprobar si con este nuevo cambio se incumple alguna restricción o si se ha dejado de incumplir una que existía previamente. Para la distancia tan solo se tendrá que recalcular la dispersión media de estos 2 clústeres y reconsiderar la suma de todos. Para que esto sea posible se genera una estructura como la siguiente:

```

1 Clase factorizacionSolucion:
2     infactibilidad                // Número de restricciones incumplidas
3     centroides                   // Vector de centroides actuales
4     n_puntos_cluster              // Vector de número de puntos por cluster
5     distancia_ic                  // Vector de distancia intra-cluster
6     desviacionGeneral             // Desviación general
7     fitness                      // Función fitness

```

Código Fuente 10: Pseudocódigo de la generación del vecindario

De esta forma, si se inicializa adecuadamente, la actualización de una solución tras asignar x_i al nuevo cluster c_{nuevo} quedaría de la siguiente forma:

```

1 funcion ActualizarFactorizacionSolucion(S, S_fac, i, c_nuevo):
2     c_anterior = S[i]
3     Si c_anterior == c_nuevo:
4         return S_fac // Si no cambia de cluster, no hacemos nada
5
6 // Actualizamos infactibilidad
7     S_fac.infactibilidad = ActualizarInfactibilidad(S, S_fac, i, c_nuevo)
8
9 // Actualizamos centroides
10    n_anterior = S_info.n_puntos_cluster[c_anterior]
11    n_nuevo = S_info.n_puntos_cluster[c_nuevo]
12
13 // Extraemos el punto del centroide anterior: (Media_anterior * N - Punto) /
14 // (N - 1)
15 Para cada d desde 0 hasta la dimensión de X[i]:
16     suma_anterior = S_fac.centroides[c_anterior].coordenada[d] * n_anterior
17     S_info.centroides[c_anterior].coordenada[d] = (suma_anterior - X[i].
18     coordenada[d]) / (n_anterior - 1);
19
20 // Añadimos el punto al centroide nuevo: (Media_vieja * N + Punto) / (N + 1)
21 Para cada d desde 0 hasta la dimensión de X[i]:
22     suma_anterior = S_fac.centroides[c_nuevo].coordenada[d] * n_nuevo
23     S_info.centroides[c_nuevo].coordenada[d] = (suma_anterior + X[i].
24     coordenada[d]) / (n_nuevo + 1);
25
26 // Actualizamos la cantidad de puntos
27     S_fac.n_puntos_cluster[c_anterior]--
28     S_fac.n_puntos_cluster[c_nuevo]++
29
30 // Actualizamos la distancia intra-cluster
31 // Como el centroide se movió, recalculamos las distancias solo de los
32 // puntos en c_anterior y c_nuevo
33 anterior_dist_ic_anterior = S_fac.distancia_ic[c_anterior]
34 anterior_dist_ic_nueva = S_fac.distancia_ic[c_nuevo]
35
36 nueva_dist_ic_anterior = 0.0

```

```

33     nueva_dist_ic_nueva = 0.0
34
35     Para cada j desde 0 hasta n-1:
36         Si j==i:
37             // El punto que se mueve, ahora se mide respecto a c_nuevo
38             nueva_dist_ic_nueva += distancia(X[i], S_fac.centroides[c_nuevo])
39         Sino si S[j] == c_anterior:
40             nueva_dist_ic_anterior += distancia(X[i], S_fac.centroides[
41             c_anterior])
42         Sino si S[j] != c_nueva:
43             nueva_dist_ic_nueva += distancia(X[j], S_fac.centroides[c_nuevo])
44
45     S_fac.distancia_ic[c_anterior] = nueva_dist_ic_anterior
46     S_fac.distancia_ic[c_nuevo] = nueva_dist_ic_nueva
47
48     // Actualizamos la desviación general
49     // Media antigua de estos dos clústeres
50     anterior_contrib_anterior = Si (n_anterior > 0) entonces (
51     anterior_dist_ic_anterior / n_anterior) Sino 0.0
52     anterior_contrib_nueva = Si (n_nuevo > 0) entonces (anterior_dist_ic_nueva /
53     n_nuevo) Sino 0.0
54
55     // Media nueva de estos dos clústeres
56     nuevo_n_anterior = S_fac.n_puntos_cluster[c_anterior];
57     nuevo_n_nuevo = S_fac.n_puntos_cluster[c_nuevo];
58     nueva_contrib_anterior = Si (nuevo_n_anterior > 0) entonces (
59     nueva_dist_ic_anterior / nuevo_n_anterior) Sino 0.0
60     nueva_contrib_nueva = Si (nuevo_n_nuevo > 0) entonces (nueva_dist_ic_nueva /
61     nuevo_n_nuevo) Sino 0.0
62
63     // Fórmula para actualizar la media global en O(1)
64     sum_desv = (S_fac.desviacionGeneral * k) - anterior_contrib_anterior -
65     anterior_contrib_nueva + nueva_contrib_anterior + nueva_contrib_nueva
66
67     S_fac.desviacionGeneral = sum_desv / k;
68
69     // Actualizamos el fitness
70     S_fac.fitness = S_fac.desviacionGeneral + S.lambda * S_fac.infactibilidad
71
72     return S_fac

```

Código Fuente 11: Pseudocódigo de la actualización de una solución factorizada

De esta forma los cálculos son mínimos y muy rápidos. De esta forma, evaluar una solución modificada consistirá en

```

1  funcion Evaluacion(S, S_fac, i, c_nuevo):
2      // Si no hay cambio, calculamos el fitness con la info actual directamente
3      Si (S[i] == c_nuevo) return S_fac.desviacionGeneral + S.lambda * S_fac.
4      infactibilidad
5
6      // Si cambia actualizamos una copia (para no modificarlo)
7      nueva_fac = ActualizarFactorizacionSolucion(S, S_fac, i, c_nuevo)
8
9      return nueva_fac.desviacionGeneral + S.lambda * nueva_fac.infactibilidad

```

Código Fuente 12: Pseudocódigo de la evaluación de una solución factorizada

3.3.3. Estructura del código de Búsqueda Local

Con las herramientas anteriores, el algoritmo queda:

```

1  funcion optimizarBL(maxevals):
2
3      // Generamos una solución inicial válida
4      S = generarSolucionAleatoria()
5
6      // Nos aseguramos de que es válida
7      Si S no es válida:

```

```

8      S = RepararSolucion(S)
9
10     S_fac = Factorizar(S)
11     evals = 1
12
13     // Evaluamos la solución inicial
14     mejor_fitness = S_fac.fitness;
15
16     mejora = true;
17
18     // Bucle principal
19     Mientras (mejora y evals < maxevals):
20         mejora = false
21
22         // Generamos y barajamos el vecindario actual
23         vecindario = generarVecindario(S)
24         barajar(vecindario)
25
26         // Exploramos el vecindario
27         Para cada cambio m en el vecindario:
28             Si (evals >= maxevals):
29                 break
30
31             i = m.punto
32             j = m.c_nuevo
33
34             // Evaluamos el vecino
35             fitness_vecino = Evaluacion(S, S_fac, i, j)
36             evals++
37
38             // Si encontramos un fitness mejor
39             Si (fitness_vecino < mejor_fitness) :
40                 // Aplicamos el movimiento inmediatamente
41                 S_fac = ActualizarFactorizacionSolucion(S, S_fac, i, j)
42                 S[i] = j;
43                 mejor_fitness = fitness_vecino;
44                 mejora = true;
45
46                 // Detenemos la exploración de este vecindario
47                 break;
48
49     return S

```

Código Fuente 13: Pseudocódigo del algoritmo de Búsqueda Local

4. Estructura general del código

El código está programado sobre la API proporcionada de C++. Los ficheros creados principales se encuentran en la carpeta `inc`. El programa principal se encuentra en el fichero `main.cpp` que se podrá ejecutar de cualquiera de las siguientes formas:

```

1  main
2  main <seed>
3  main <seed> <filename> <ratio> <k>

```

Además se ha creado un script para compilarlo y ejecutarlo, que se deberá ejecutar desde la carpeta raíz del proyecto (la inmediatamente superior a la carpeta que contiene el código). Por tanto, para compilar y ejecutar el proyecto tendremos que ejecutar desde una terminal

```

1  ./compilar_y_ejecutar.sh

```

5. Experimentos y análisis de resultados

5.1. Descripción de casos del problema

Esta implementación se ha utilizado para los siguientes casos:

1. Zoo: En donde hay que agrupar animales en función de 16 atributos (si tiene cola, . . .). Tiene 7 clases ($k = 7$) y 101 instancias.
2. Glass: En donde hay que agrupar distintos vidrios en función de 9 atributos sobre sus componentes químicos. Tiene 7 clases ($k = 7$) y 214 instancias.
3. Bupa: En donde hay que agrupar personas en función de sus hábitos de consumo de alcohol, definido con 5 atributos. Tiene 16 clases ($k = 16$) y 345 instancias.

Más adelante se especificarán el resto de parámetros

5.2. Resultados obtenidos

Se ha ejecutado una batería de pruebas con un programa python (`metrics.py`), ejecutando cada algoritmo para cada *dataset* y *ratio* para 10 semillas distintas, en concreto las siguientes:

```
1 SEEDS = [12345, 67890, 13579, 24680, 98765, 43210, 10293, 84756, 97531, 8642]
```

Código Fuente 14: Semillas utilizadas para las pruebas

Tras ejecutar el programa se han extraído las siguientes métricas:

Algoritmo	Dataset	Ratio	Fitness		Distancia		Incumplimiento		Evals	Tiempo (ms)	
			Media	Std	Media	Std	Media	Std		Media	Std
RandomSearch	bupa	15	0.3284	0.0011	0.2479	0.0015	0.0533	0.0009	100000.0	1680.2	131.6078
RandomSearch	bupa	30	0.3335	0.0007	0.2469	0.0020	0.0574	0.0013	100000.0	2823.7	348.8878
RandomSearch	glass	15	0.6796	0.0035	0.4289	0.0051	0.1176	0.0028	100000.0	865.7	62.3628
RandomSearch	glass	30	0.6968	0.0023	0.4214	0.0037	0.1292	0.0018	100000.0	1378.4	144.3139
RandomSearch	zoo	15	1.9318	0.0133	1.5668	0.0620	0.0991	0.0164	100000.0	444.4	46.1909
RandomSearch	zoo	30	1.9693	0.0197	1.5049	0.0821	0.1261	0.0213	100000.0	484.4	13.9459
Greedy	bupa	15	0.2400	0.0024	0.2390	0.0021	0.0007	0.0004	1.0	326.2	52.3913
Greedy	bupa	30	0.2676	0.0025	0.2476	0.0025	0.0133	0.0005	1.0	695.5	99.2765
Greedy	glass	15	0.4880	0.0063	0.4276	0.0039	0.0283	0.0024	1.0	24.9	2.9231
Greedy	glass	30	0.5592	0.0065	0.4259	0.0050	0.0625	0.0015	1.0	63.7	16.2894
Greedy	zoo	15	1.5067	0.0290	1.4973	0.0293	0.0026	0.0014	1.0	2.1	0.3162
Greedy	zoo	30	1.7169	0.0313	1.5996	0.0251	0.0318	0.0030	1.0	4.4	1.6465
LocalSearch	bupa	15	0.1702	0.0023	0.1437	0.0028	0.0175	0.0008	82526.6	710.5	147.5144
LocalSearch	bupa	30	0.1982	0.0024	0.1437	0.0040	0.0361	0.0014	81769.3	1310.2	293.4745
LocalSearch	glass	15	0.4041	0.0063	0.3037	0.0074	0.0471	0.0025	16262.5	70.3	15.9168
LocalSearch	glass	30	0.4682	0.0038	0.2862	0.0039	0.0854	0.0028	17931.4	135.2	32.2621
LocalSearch	zoo	15	1.0923	0.0368	0.9492	0.0559	0.0389	0.0110	6201.6	11.6	1.8379
LocalSearch	zoo	30	1.1968	0.0327	0.8713	0.0642	0.0884	0.0106	6201.2	14.5	3.9791

Tabla 1: Resumen de métricas de rendimiento por algoritmo, conjunto de datos y ratio de restricción para 10 semillas distintas.

5.2.1. Tablas globales por problema

Con el objetivo de estudiar mejor este resultado rellenamos también las siguientes tablas con la información extraída de la tabla anterior

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (ms)
Greedy	1.5067	1.4973	0.0026	1.0	2.1
Random	1.9318	1.5668	0.0991	100000.0	444.4
BL	1.0923	0.9492	0.0389	6201.6	11.6

Tabla 2: Resultados obtenidos para el dataset **Zoo** con 15 % de restricciones en 10 ejecuciones independientes.

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (ms)
Greedy	1.7169	1.5996	0.0318	1.0	4.4
Random	1.9693	1.5049	0.1261	100000.0	484.4
BL	1.1968	0.8713	0.0884	6201.2	14.5

Tabla 3: Resultados obtenidos para el dataset **Zoo** con 30 % de restricciones en 10 ejecuciones independientes.

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (ms)
Greedy	0.4880	0.4276	0.0283	1.0	24.9
Random	0.6796	0.4289	0.1176	100000.0	865.7
BL	0.4041	0.3037	0.0471	16262.5	70.3

Tabla 4: Resultados obtenidos para el dataset **Glass** con 15 % de restricciones en 10 ejecuciones independientes.

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (ms)
Greedy	0.5592	0.4259	0.0625	1.0	63.7
Random	0.6968	0.4214	0.1292	100000.0	1378.4
BL	0.4682	0.2862	0.0854	17931.4	135.2

Tabla 5: Resultados obtenidos para el dataset **Glass** con 30 % de restricciones en 10 ejecuciones independientes.

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (ms)
Greedy	0.2400	0.2390	0.0007	1.0	326.2
Random	0.3284	0.2479	0.0533	100000.0	1680.2
BL	0.1702	0.1437	0.0175	82526.6	710.5

Tabla 6: Resultados obtenidos para el dataset **Bupa** con 15 % de restricciones en 10 ejecuciones independientes.

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (ms)
Greedy	0.2676	0.2476	0.0133	1.0	695.5
Random	0.3335	0.2469	0.0574	100000.0	2823.7
BL	0.1982	0.1437	0.0361	81769.3	1310.2

Tabla 7: Resultados obtenidos para el dataset **Bupa** con 30 % de restricciones en 10 ejecuciones independientes.

5.2.2. Boxplots globales por problema

Adicionalmente se han generado los siguientes gráficos (script `graphics.py`) para visualizar la calidad de cada algoritmo:

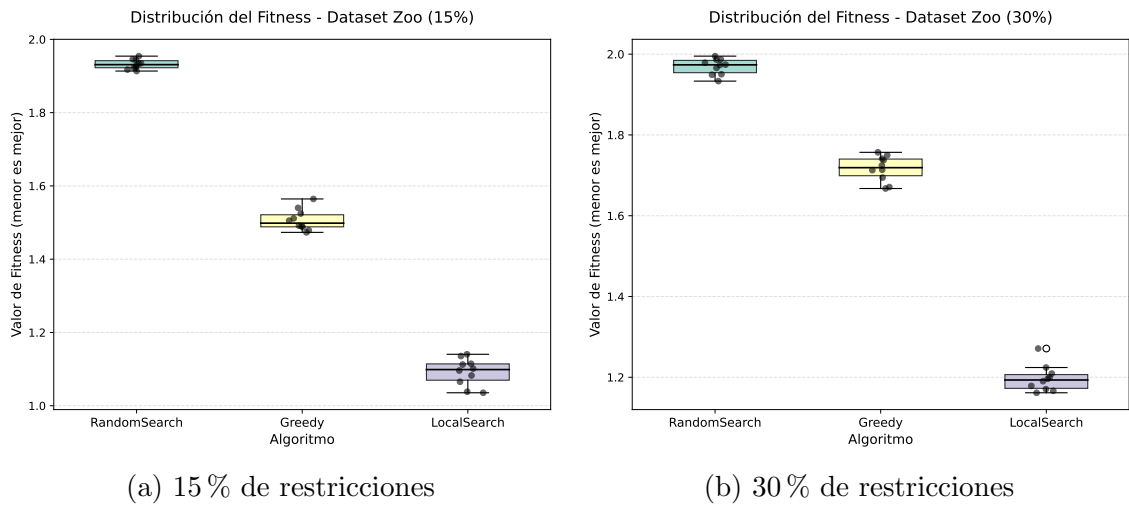


Figura 1: Distribución de los valores de Fitness obtenidos para el dataset **Zoo** en 10 ejecuciones independientes.

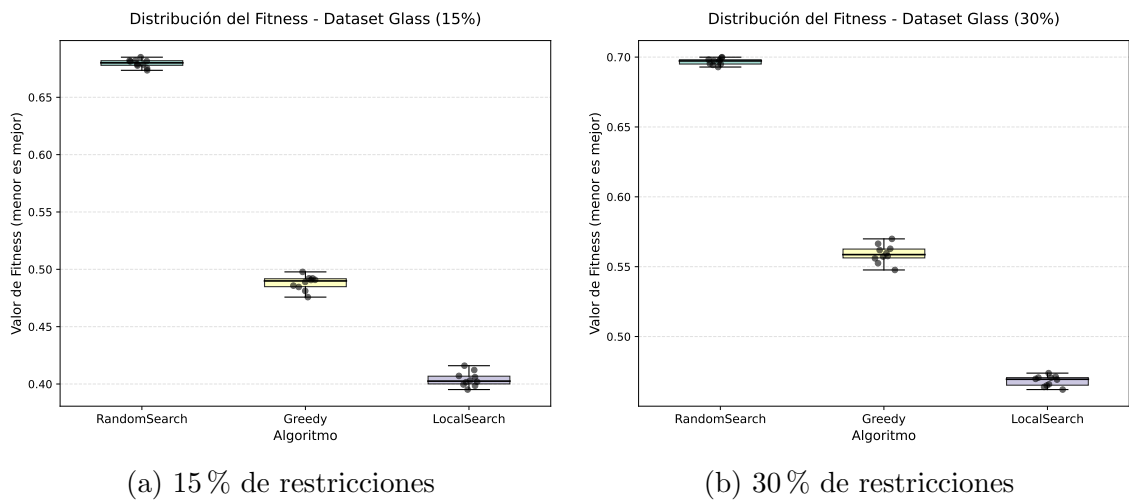


Figura 2: Distribución de los valores de Fitness obtenidos para el dataset **Glass** en 10 ejecuciones independientes.

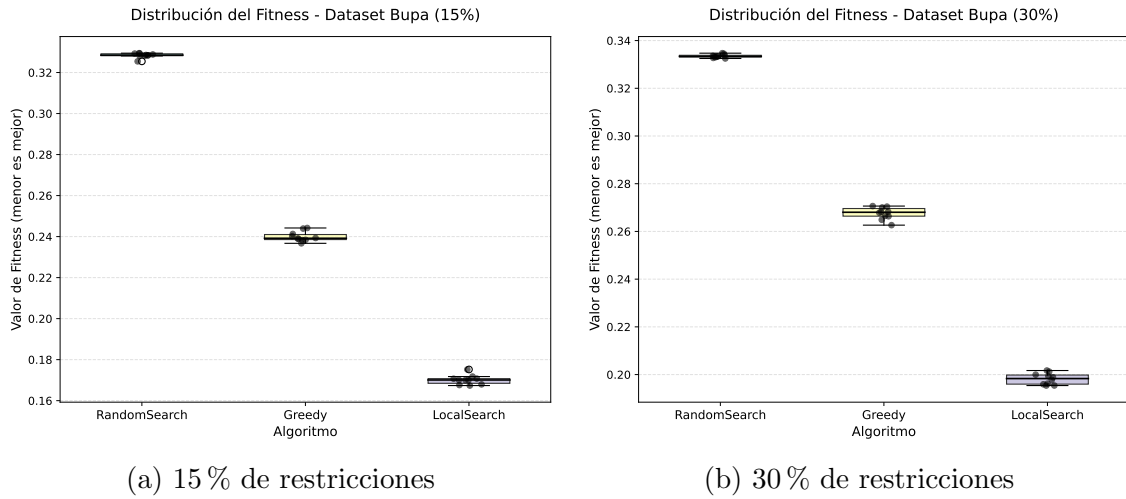


Figura 3: Distribución de los valores de Fitness obtenidos para el dataset **Bupa** en 10 ejecuciones independientes.

5.3. Análisis de resultados

A partir de las métricas obtenidas en las tablas globales y la observación de la distribución en las gráficas, se pueden extraer conclusiones muy claras sobre el rendimiento y comportamiento de los tres algoritmos evaluados. El análisis se dividirá en cuatro bloques principales: calidad de las soluciones, satisfacción de restricciones, eficiencia computacional y el impacto de la complejidad del problema.

5.3.1. Calidad de las soluciones

Comparando los resultados llegamos a las siguientes conclusiones de cada algoritmo:

- **Búsqueda Local (BL):** Ha demostrado ser el algoritmo más robusto y eficaz en términos de calidad pura. Para todos los conjuntos de datos (Zoo, Glass y Bupa) y ratios de restricción, la BL obtiene consistentemente los valores más bajos tanto de fitness general como de distancia intra-clúster. Esto demuestra que la exploración del vecindario guiada por gradiente descendente permite refinar adecuadamente las asignaciones hasta alcanzar mínimos locales de muy buena calidad.
- **Greedy:** Se posiciona como una heurística intermedia. Supera ampliamente al modelo aleatorio, logrando configuraciones razonablemente buenas, pero se estanca en óptimos locales peores que los de la Búsqueda Local, ya que su enfoque constructivo no le permite retroceder o reevaluar asignaciones previas.
- **Algoritmo Aleatorio:** Como era de esperar en espacios de búsqueda combinatorios tan amplios, el muestreo aleatorio puro (incluso con 100.000 iteraciones) ofrece resultados no tan satisfactorios. Presenta la mayor dispersión y los peores valores de *fitness* (nos sirve para ver que efectivamente los otros algoritmos funcionan bien).

5.3.2. Satisfacción de restricciones

Si estudiamos el número de restricciones incumplidas llegamos a que:

-) El algoritmo **Greedy** es el claro ganador en el cumplimiento de las restricciones (valores más cercanos a 0 en todas las configuraciones). Al evaluar de manera determinista la violación exacta en cada inserción, el algoritmo logra evitar casi por completo incumplir las restricciones
-) Por el contrario, la **Búsqueda Local** sacrifica este factor. Dado que la función objetivo pondera la distancia y las restricciones mediante el parámetro λ , la BL encuentra que, en ciertas ocasiones, es mejor (globalmente) violar una restricción para agrupar elementos geoméricamente muy cercanos, reduciendo la distancia a cambio de una pequeña penalización en la infactibilidad.

5.3.3. Eficiencia computacional

El análisis del eficiencia computacional también es interesante:

-) **Aleatorio vs. Búsqueda Local:** El algoritmo Aleatorio consume siempre el máximo de evaluaciones (100.000) de forma bruta. Esto se traduce en tiempos altos (ej. hasta 2823 ms en Bupa 30 %). Por su parte, la BL realiza un número variable de evaluaciones hasta converger (entre 6.200 en Zoo y 82.000 en Bupa). Sin embargo, gracias a la evaluación factorizada, los tiempos de la BL son sistemáticamente inferiores a los del algoritmo aleatorio (ej. 1310 ms en Bupa 30 %), demostrando que el diseño de estructuras de datos eficientes compensa con creces el coste de generar vecindarios.
-) **Greedy:** Es indiscutiblemente el algoritmo más rápido en términos absolutos ya que construye la solución en una sola pasada (registrando una única evaluación en cada caso).

5.3.4. Impacto de la escalabilidad y las restricciones

Finalmente, los experimentos evidencian la sensibilidad de los algoritmos ante el aumento de la complejidad:

-) **Aumento del Ratio de Restricciones (15 % a 30 %):** Al duplicar las imposiciones sobre el espacio geométrico, la dificultad del problema aumenta de forma evidente. En los tres datasets se observa que pasar al 30 % provoca un empeoramiento del fitness global, un incremento en la tasa de incumplimiento y un ligero aumento en los tiempos de ejecución.
-) **Tamaño del Dataset:** Al transicionar de Zoo (101 instancias) a Glass (214) y Bupa (345), el tiempo de ejecución crece bastante (especialmente de Glass a Bupa, donde también pasamos de $k = 7$ a $k = 16$). La Búsqueda Local requiere explorar vecindarios mucho más grandes, aumentando sus tiempos y las evaluaciones requeridas para llegar a un óptimo local (de unas 6.000 en Zoo a más de 80.000 en Bupa).

Conclusión general: La **Búsqueda Local** con evaluación factorizada es sin ninguna duda el mejor método para este problema. Consigue el mejor equilibrio entre cercanía de los puntos de los clústeres, tolerancia al cumplimiento de restricciones y eficiencia temporal. El algoritmo **Greedy** podría servir de semilla inicial para algoritmos más complejos, dada su alta rapidez y factibilidad.